

ДИСЦИПЛИНА	Программирование в операционной системе Линукс
ИНСТИТУТ	Передовая инженерная школа СВЧ-электроники
КАФЕДРА	Передовых технологий
ВИД УЧЕБНОГО МАТЕРИАЛА	Методические указания по дисциплине
ПРЕПОДАВАТЕЛЬ	Астафьев Рустам Уралович
СЕМЕСТР	1 семестр, 2025/2026 уч. год

Ссылка на материал:
<https://github.com/astafiev-rustam/programming-in-the-linux-operating-system/tree/lecture-1-6>

Лекция №6: Создание и использование разделяемых библиотек

Теоретическая вводная

Разделяемые библиотеки: искусство совместного использования кода

Представьте, что вы живете в многоквартирном доме, где есть общие помещения — библиотека, спортзал, бассейн. Все жители могут пользоваться этими помещениями, не создавая их копии в каждой квартире. Именно так работают разделяемые (динамические) библиотеки в Linux. Это коллекции функций и данных, которые могут использоваться множеством программ одновременно, экономя память и дисковое пространство.

В отличие от статических библиотек, которые встраиваются в исполняемый файл на этапе компиляции, разделяемые библиотеки остаются отдельными файлами и загружаются в память только тогда, когда программа их запрашивает. Это создает мощную экосистему, где обновление библиотеки автоматически становится доступным для всех программ, которые ее используют — как если бы вы обновили оборудование в общем спортзале, и все жители сразу получили доступ к улучшениям.

Механизм работы: от компиляции до выполнения

Когда вы компилируете программу с использованием разделяемой библиотеки, компилятор не включает код библиотеки в исполняемый файл. Вместо этого он оставляет специальные метки — ссылки на функции из библиотеки. Эти метки говорят системе: "Когда программа будет запущена, найди вот эту библиотеку и подключи эти функции".

Ключевой момент здесь — флаг `-fPIC` (Position Independent Code). Он заставляет компилятор генерировать код, который может работать независимо от того, в каком месте памяти он будет загружен. Это критически важно, потому что система не может гарантировать, что библиотека будет загружена по одному и тому же адресу в памяти для разных программ.

Динамический загрузчик: мост между программой и библиотекой

Когда вы запускаете программу, в игру вступает динамический загрузчик (`ld.so`) — умный посредник, который знает, где искать нужные библиотеки. Он проверяет зависимости программы, находит соответствующие файлы библиотек в файловой системе и загружает их в память.

Загрузчик следует четкому алгоритму поиска: сначала он проверяет пути, указанные в переменной окружения `LD_LIBRARY_PATH`, затем смотрит в кэше библиотек (`/etc/ld.so.cache`), и наконец проверяет стандартные системные директории (`/lib`, `/usr/lib`). Этот многоуровневый подход обеспечивает гибкость — вы можете временно переопределить путь к библиотеке для тестирования, не затрагивая системную конфигурацию.

Версионирование: эволюция без разрушения

В мире разделяемых библиотек версионирование — это не просто формальность, а необходимость для поддержания совместимости. Библиотеки используют систему версий в своих именах:

`libname.so.1.2.3`, где первая цифра — major version (меняется при несовместимых изменениях), вторая — minor version (новый функционал с обратной совместимостью), третья — patch version (исправления ошибок).

Система симлинков создает элегантную структуру: `libname.so` указывает на `libname.so.1`, который указывает на `libname.so.1.2.3`. Программы линкуются против `libname.so`, но загружают конкретную версию `libname.so.1.2.3`. Это позволяет иметь несколько версий библиотеки одновременно и гарантирует, что программы получат именно ту версию, с которой они были скомпилированы.

Отладка и диагностика: понимание зависимостей

Работа с разделяемыми библиотеками требует умения диагностировать проблемы. Утилита `ldd` показывает, от каких библиотек зависит программа и где система их найдет. `nm` позволяет заглянуть внутрь библиотеки и увидеть, какие символы (функции и переменные) она экспортирует. `objdump` предоставляет еще более детальную информацию о структуре библиотеки.

Когда что-то идет не так — библиотека не находится или версия не совпадает — эти инструменты становятся вашими главными помощниками в решении проблемы. Они позволяют понять, какая именно библиотека требуется, какая версия установлена и почему система не может ее найти.

Практические примеры

Пример 1: Создание и использование простой разделяемой библиотеки

Цель: Создать базовую разделяемую библиотеку и использовать ее в программе.

```
# 1. Создаем заголовочный файл библиотеки
cat > stringlib.h << 'EOF'
#ifndef STRINGLIB_H
#define STRINGLIB_H

// Функция для переворота строки
void reverse_string(char* str);

// Функция для преобразования строки в верхний регистр
```

```
void to_uppercase(char* str);

// Функция для проверки, является ли строка палиндромом
int is_palindrome(const char* str);

// Функция для подсчета количества слов в строке
int count_words(const char* str);

#endif
EOF

# 2. Создаем реализацию библиотеки
cat > stringlib.c << 'EOF'
#include "stringlib.h"
#include <ctype.h>
#include <string.h>
#include <stdio.h>

void reverse_string(char* str) {
    if (str == NULL) return;

    int len = strlen(str);
    for (int i = 0; i < len / 2; i++) {
        char temp = str[i];
        str[i] = str[len - i - 1];
        str[len - i - 1] = temp;
    }
}

void to_uppercase(char* str) {
    if (str == NULL) return;

    for (int i = 0; str[i]; i++) {
        str[i] = toupper(str[i]);
    }
}

int is_palindrome(const char* str) {
    if (str == NULL) return 0;

    int len = strlen(str);
    for (int i = 0; i < len / 2; i++) {
        if (str[i] != str[len - i - 1]) {
            return 0;
        }
    }
    return 1;
}

int count_words(const char* str) {
    if (str == NULL) return 0;

    int count = 0;
    int in_word = 0;
```

```
    for (int i = 0; str[i]; i++) {
        if (isspace(str[i])) {
            in_word = 0;
        } else if (!in_word) {
            in_word = 1;
            count++;
        }
    }
}
return count;
}
EOF
```

3. Компилируем с позиционно-независимым кодом (обязательно для shared libraries)
gcc -c -fPIC stringlib.c -o stringlib.o

4. Создаем разделяемую библиотеку
gcc -shared -o libstringlib.so stringlib.o

5. Создаем программу, которая использует нашу библиотеку

```
cat > main.c << 'EOF'
#include <stdio.h>
#include <stdlib.h>
#include "stringlib.h"

int main() {
    char text[100];

    printf("Введите строку: ");
    fgets(text, sizeof(text), stdin);

    // Убираем символ новой строки
    text[strcspn(text, "\n")] = 0;

    printf("Исходная строка: %s\n", text);

    // Используем функции из нашей библиотеки
    char reversed[100];
    strcpy(reversed, text);
    reverse_string(reversed);
    printf("Перевернутая: %s\n", reversed);

    char upper[100];
    strcpy(upper, text);
    to_uppercase(upper);
    printf("В верхнем регистре: %s\n", upper);

    printf("Палиндром: %s\n", is_palindrome(text) ? "да" : "нет");
    printf("Количество слов: %d\n", count_words(text));

    return 0;
}
EOF
```

```
# 6. Компилируем программу, линкуясь с нашей библиотекой
gcc main.c -L. -lstringlib -o stringdemo

# 7. Пытаемся запустить (пока не получится - библиотека не найдена)
./stringdemo || echo "Библиотека не найдена!"

# 8. Смотрим зависимости программы
ldd stringdemo

# 9. Временно добавляем текущую директорию в путь поиска библиотек
export LD_LIBRARY_PATH=.:$LD_LIBRARY_PATH

# 10. Теперь программа должна работать
./stringdemo
```

Пример 2: Установка библиотеки в систему и управление кэшем

Цель: Научиться устанавливать библиотеку в системные директории и управлять кэшем библиотек.

```
# 1. Создаем более сложную библиотеку с математическими функциями
cat > mathlib.h << 'EOF'
#ifndef MATHLIB_H
#define MATHLIB_H

// Базовые математические операции
double add(double a, double b);
double subtract(double a, double b);
double multiply(double a, double b);
double divide(double a, double b);

// Статистические функции
double calculate_mean(const double* data, int count);
double calculate_stddev(const double* data, int count);

// Утилиты
void print_statistics(const double* data, int count);

#endif
EOF

cat > mathlib.c << 'EOF'
#include "mathlib.h"
#include <stdio.h>
#include <math.h>

double add(double a, double b) {
    return a + b;
}

double subtract(double a, double b) {
    return a - b;
}
```

```
}

double multiply(double a, double b) {
    return a * b;
}

double divide(double a, double b) {
    if (b == 0.0) {
        fprintf(stderr, "Ошибка: деление на ноль!\n");
        return 0.0;
    }
    return a / b;
}

double calculate_mean(const double* data, int count) {
    if (data == NULL || count <= 0) return 0.0;

    double sum = 0.0;
    for (int i = 0; i < count; i++) {
        sum += data[i];
    }
    return sum / count;
}

double calculate_stddev(const double* data, int count) {
    if (data == NULL || count <= 1) return 0.0;

    double mean = calculate_mean(data, count);
    double sum_sq = 0.0;

    for (int i = 0; i < count; i++) {
        double diff = data[i] - mean;
        sum_sq += diff * diff;
    }

    return sqrt(sum_sq / (count - 1));
}

void print_statistics(const double* data, int count) {
    if (data == NULL || count <= 0) {
        printf("Нет данных для анализа\n");
        return;
    }

    double mean = calculate_mean(data, count);
    double stddev = calculate_stddev(data, count);

    printf("Статистика:\n");
    printf("  Количество элементов: %d\n", count);
    printf("  Среднее значение: %.2f\n", mean);
    printf("  Стандартное отклонение: %.2f\n", stddev);

    // Находим минимум и максимум
    double min = data[0];
```

```
double max = data[0];
for (int i = 1; i < count; i++) {
    if (data[i] < min) min = data[i];
    if (data[i] > max) max = data[i];
}
printf("  Минимум: %.2f\n", min);
printf("  Максимум: %.2f\n", max);
}
EOF

# 2. Компилируем и создаем библиотеку с версией
gcc -c -fPIC mathlib.c -o mathlib.o
gcc -shared -Wl,-soname,libmathlib.so.1 -o libmathlib.so.1.0.0 mathlib.o -lm

# 3. Создаем симлинки для совместимости
ln -sf libmathlib.so.1.0.0 libmathlib.so.1
ln -sf libmathlib.so.1 libmathlib.so

# 4. Проверяем созданные файлы
ls -la libmathlib.so*

# 5. Смотрим информацию о библиотеке
objdump -p libmathlib.so.1.0.0 | grep SONAME

# 6. Устанавливаем библиотеку в систему
sudo mkdir -p /usr/local/include /usr/local/lib
sudo cp mathlib.h /usr/local/include/
sudo cp libmathlib.so.1.0.0 /usr/local/lib/
sudo cp -P libmathlib.so.1 /usr/local/lib/
sudo cp -P libmathlib.so /usr/local/lib/

# 7. Обновляем кэш библиотек
sudo ldconfig

# 8. Проверяем, что библиотека добавлена в кэш
ldconfig -p | grep mathlib

# 9. Создаем тестовую программу
cat > mathtest.c << 'EOF'
#include <stdio.h>
#include "mathlib.h"

int main() {
    double numbers[] = {1.5, 2.8, 3.2, 4.7, 5.1};
    int count = sizeof(numbers) / sizeof(numbers[0]);

    printf("Математическая библиотека v1.0\n\n");

    // Базовые операции
    printf("Базовые операции:\n");
    printf("  5.2 + 3.1 = %.2f\n", add(5.2, 3.1));
    printf("  7.8 - 2.3 = %.2f\n", subtract(7.8, 2.3));
    printf("  2.5 * 4.0 = %.2f\n", multiply(2.5, 4.0));
    printf("  9.0 / 2.0 = %.2f\n\n", divide(9.0, 2.0));
```

```
// Статистика
printf("Анализ данных:\n");
print_statistics(numbers, count);

return 0;
}
EOF

# 10. Компилируем программу (теперь библиотека находится в стандартном пути)
gcc mathtest.c -lmathlib -o mathtest

# 11. Запускаем программу (работает без LD_LIBRARY_PATH!)
./mathtest
```

Пример 3: Продвинутое версионирование и обратная совместимость

Цель: Создать несколько версий библиотеки и обеспечить обратную совместимость.

```
# 1. Создаем первую версию библиотеки (v1.0)
cat > configlib_v1.h << 'EOF'
#ifndef CONFIGLIB_V1_H
#define CONFIGLIB_V1_H

// Конфигурационная библиотека v1.0
typedef struct {
    char name[50];
    int timeout;
    int max_connections;
} Config_v1;

void config_init_v1(Config_v1* config);
void config_print_v1(const Config_v1* config);
int config_validate_v1(const Config_v1* config);

#endif
EOF

cat > configlib_v1.c << 'EOF'
#include "configlib_v1.h"
#include <stdio.h>
#include <string.h>

void config_init_v1(Config_v1* config) {
    if (config == NULL) return;

    strcpy(config->name, "default");
    config->timeout = 30;
    config->max_connections = 100;
}
```



```

void config_print_v1(const Config_v1* config) {
    if (config == NULL) return;

    printf("=== Конфигурация v1.0 ===\n");
    printf("Имя: %s\n", config->name);
    printf("Таймаут: %d сек\n", config->timeout);
    printf("Макс. подключений: %d\n", config->max_connections);
}

int config_validate_v1(const Config_v1* config) {
    if (config == NULL) return 0;

    if (config->timeout <= 0) {
        printf("Ошибка: таймаут должен быть положительным\n");
        return 0;
    }

    if (config->max_connections <= 0) {
        printf("Ошибка: максимальное количество подключений должно быть
положительным\n");
        return 0;
    }

    return 1;
}
EOF

```

2. Создаем вторую версию библиотеки (v2.0) с новыми функциями

```
cat > configlib_v2.h << 'EOF'
```

```
#ifndef CONFIGLIB_V2_H
```

```
#define CONFIGLIB_V2_H
```

```
// Конфигурационная библиотека v2.0
```

```
// Новая версия с дополнительными полями
```

```
typedef struct {
```

```
    char name[50];
```

```
    int timeout;
```

```
    int max_connections;
```

```
    int cache_size;        // Новое поле в v2.0
```

```
    int enable_logging;    // Новое поле в v2.0
```

```
} Config_v2;
```

```
void config_init_v2(Config_v2* config);
```

```
void config_print_v2(const Config_v2* config);
```

```
int config_validate_v2(const Config_v2* config);
```

```
void config_set_logging(Config_v2* config, int enable); // Новая функция в v2.0
```

```
#endif
```

```
EOF
```

```
cat > configlib_v2.c << 'EOF'
```

```
#include "configlib_v2.h"
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
void config_init_v2(Config_v2* config) {
    if (config == NULL) return;

    strcpy(config->name, "default");
    config->timeout = 30;
    config->max_connections = 100;
    config->cache_size = 1024;        // Значение по умолчанию для нового поля
    config->enable_logging = 1;       // Значение по умолчанию для нового поля
}

void config_print_v2(const Config_v2* config) {
    if (config == NULL) return;

    printf("=== Конфигурация v2.0 ===\n");
    printf("Имя: %s\n", config->name);
    printf("Таймаут: %d сек\n", config->timeout);
    printf("Макс. подключений: %d\n", config->max_connections);
    printf("Размер кэша: %d MB\n", config->cache_size);
    printf("Логирование: %s\n", config->enable_logging ? "включено" :
"выключено");
}

int config_validate_v2(const Config_v2* config) {
    if (config == NULL) return 0;

    if (config->timeout <= 0) {
        printf("Ошибка: таймаут должен быть положительным\n");
        return 0;
    }

    if (config->max_connections <= 0) {
        printf("Ошибка: максимальное количество подключений должно быть
положительным\n");
        return 0;
    }

    if (config->cache_size < 0) {
        printf("Ошибка: размер кэша не может быть отрицательным\n");
        return 0;
    }

    return 1;
}

void config_set_logging(Config_v2* config, int enable) {
    if (config == NULL) return;
    config->enable_logging = enable;
}

EOF
```

```
# 3. Компилируем обе версии библиотеки
# Версия 1.0
gcc -c -fPIC configlib_v1.c -o configlib_v1.o
```

```
gcc -shared -Wl,-soname,libconfiglib.so.1 -o libconfiglib.so.1.0.0 configlib_v1.o

# Версия 2.0
gcc -c -fPIC configlib_v2.c -o configlib_v2.o
gcc -shared -Wl,-soname,libconfiglib.so.2 -o libconfiglib.so.2.0.0 configlib_v2.o

# 4. Создаем симлинки
ln -sf libconfiglib.so.1.0.0 libconfiglib.so.1
ln -sf libconfiglib.so.2.0.0 libconfiglib.so.2
ln -sf libconfiglib.so.1 libconfiglib.so # По умолчанию используем v1

# 5. Создаем программу, скомпилированную с v1.0
cat > app_v1.c << 'EOF'
#include <stdio.h>
#include "configlib_v1.h"

int main() {
    printf("Приложение, скомпилированное с configlib v1.0\n");

    Config_v1 config;
    config_init_v1(&config);

    // Настраиваем параметры
    strcpy(config.name, "myapp_v1");
    config.timeout = 60;
    config.max_connections = 200;

    config_print_v1(&config);

    if (config_validate_v1(&config)) {
        printf("Конфигурация валидна!\n");
    } else {
        printf("Конфигурация невалидна!\n");
    }

    return 0;
}
EOF

# 6. Создаем программу, скомпилированную с v2.0
cat > app_v2.c << 'EOF'
#include <stdio.h>
#include "configlib_v2.h"

int main() {
    printf("Приложение, скомпилированное с configlib v2.0\n");

    Config_v2 config;
    config_init_v2(&config);

    // Настраиваем параметры
    strcpy(config.name, "myapp_v2");
    config.timeout = 60;
    config.max_connections = 200;
```

```

    config.cache_size = 2048;          // Используем новое поле из v2.0
    config_set_logging(&config, 0); // Используем новую функцию из v2.0

    config_print_v2(&config);

    if (config_validate_v2(&config)) {
        printf("Конфигурация валидна!\n");
    } else {
        printf("Конфигурация невалидна!\n");
    }

    return 0;
}
EOF

# 7. Компилируем обе программы
gcc app_v1.c -L. -lconfiglib -o app_v1
gcc app_v2.c -L. -lconfiglib -o app_v2

# 8. Тестируем с разными версиями библиотеки
export LD_LIBRARY_PATH=.:$LD_LIBRARY_PATH

echo "=== Тест с версией 1.0 ==="
ln -sf libconfiglib.so.1 libconfiglib.so
./app_v1
./app_v2 || echo "app_v2 не работает с v1.0 (ожидаемо)"

echo -e "\n=== Тест с версией 2.0 ==="
ln -sf libconfiglib.so.2 libconfiglib.so
./app_v1
./app_v2

# 9. Смотрим зависимости программ
echo -e "\n=== Зависимости app_v1 ==="
ldd app_v1 | grep configlib

echo -e "\n=== Зависимости app_v2 ==="
ldd app_v2 | grep configlib

```

Пример 4: Динамическая загрузка библиотек во время выполнения

Цель: Научиться загружать библиотеки и использовать их функции во время выполнения программы.

```

# 1. Создаем библиотеку с плагинами
cat > pluginlib.h << 'EOF'
#ifndef PLUGINLIB_H
#define PLUGINLIB_H

typedef struct {
    char name[50];
    char version[20];

```

```
void (*initialize)(void);
void (*process)(const char* data);
void (*cleanup)(void);
} Plugin;

// Функции для работы с плагинами
Plugin* load_plugin(const char* plugin_name);
void unload_plugin(Plugin* plugin);

#endif
EOF

cat > pluginlib.c << 'EOF'
#include "pluginlib.h"
#include <stdio.h>
#include <string.h>
#include <dlfcn.h>

Plugin* load_plugin(const char* plugin_name) {
    if (plugin_name == NULL) return NULL;

    // Формируем имя файла библиотеки
    char libname[100];
    snprintf(libname, sizeof(libname), "./lib%s.so", plugin_name);

    // Загружаем библиотеку
    void* handle = dlopen(libname, RTLD_LAZY);
    if (!handle) {
        fprintf(stderr, "Ошибка загрузки плагина %s: %s\n", plugin_name,
            dlerror());
        return NULL;
    }

    // Создаем структуру плагина
    Plugin* plugin = malloc(sizeof(Plugin));
    if (!plugin) {
        dlclose(handle);
        return NULL;
    }

    // Загружаем функции из библиотеки
    void (*get_name)(char*) = dlsym(handle, "get_plugin_name");
    void (*get_version)(char*) = dlsym(handle, "get_plugin_version");
    void (*init_func)(void) = dlsym(handle, "plugin_initialize");
    void (*process_func)(const char*) = dlsym(handle, "plugin_process");
    void (*cleanup_func)(void) = dlsym(handle, "plugin_cleanup");

    if (!get_name || !get_version || !init_func || !process_func || !cleanup_func)
    {
        fprintf(stderr, "Ошибка: не все функции найдены в плагине %s\n",
            plugin_name);
        free(plugin);
        dlclose(handle);
        return NULL;
    }
}
```

```
}

// Заполняем структуру
get_name(plugin->name);
get_version(plugin->version);
plugin->initialize = init_func;
plugin->process = process_func;
plugin->cleanup = cleanup_func;

// Сохраняем handle для последующего закрытия
plugin->handle = handle;

return plugin;
}

void unload_plugin(Plugin* plugin) {
    if (plugin == NULL) return;

    if (plugin->handle) {
        dlclose(plugin->handle);
    }
    free(plugin);
}
EOF

# 2. Создаем первый плагин
cat > plugin_a.c << 'EOF'
#include <stdio.h>
#include <string.h>

void get_plugin_name(char* name) {
    strcpy(name, "TextProcessor");
}

void get_plugin_version(char* version) {
    strcpy(version, "1.0");
}

void plugin_initialize(void) {
    printf("Плагин TextProcessor инициализирован\n");
}

void plugin_process(const char* data) {
    printf("Обработка текста: ");
    for (int i = 0; data[i]; i++) {
        if (data[i] >= 'a' && data[i] <= 'z') {
            putchar(data[i] - 32); // В верхний регистр
        } else {
            putchar(data[i]);
        }
    }
    printf("\n");
}
```

```
void plugin_cleanup(void) {
    printf("Плагин TextProcessor завершил работу\n");
}
EOF

# 3. Создаем второй плагин
cat > plugin_b.c << 'EOF'
#include <stdio.h>
#include <string.h>

void get_plugin_name(char* name) {
    strcpy(name, "Calculator");
}

void get_plugin_version(char* version) {
    strcpy(version, "1.1");
}

void plugin_initialize(void) {
    printf("Плагин Calculator инициализирован\n");
}

void plugin_process(const char* data) {
    int a, b;
    char op;
    if (sscanf(data, "%d %c %d", &a, &op, &b) == 3) {
        int result = 0;
        switch (op) {
            case '+': result = a + b; break;
            case '-': result = a - b; break;
            case '*': result = a * b; break;
            case '/':
                if (b != 0) result = a / b;
                else { printf("Ошибка: деление на ноль\n"); return; }
                break;
            default: printf("Неизвестная операция: %c\n", op); return;
        }
        printf("Результат: %d %c %d = %d\n", a, op, b, result);
    } else {
        printf("Неверный формат данных. Используйте: число операция число\n");
    }
}

void plugin_cleanup(void) {
    printf("Плагин Calculator завершил работу\n");
}
EOF

# 4. Компилируем плагины как разделяемые библиотеки
gcc -c -fPIC plugin_a.c -o plugin_a.o
gcc -shared -o libplugin_a.so plugin_a.o

gcc -c -fPIC plugin_b.c -o plugin_b.o
gcc -shared -o libplugin_b.so plugin_b.o
```

```
# 5. Компилируем основную библиотеку для работы с плагинами
gcc -c -fPIC pluginlib.c -o pluginlib.o
gcc -shared -o libpluginlib.so pluginlib.o -ldl

# 6. Создаем основную программу, которая использует динамическую загрузку
cat > plugin_manager.c << 'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "pluginlib.h"

int main() {
    printf("=== Менеджер плагинов ===\n\n");

    // Доступные плагины
    const char* plugins[] = {"plugin_a", "plugin_b", NULL};

    // Загружаем и тестируем каждый плагин
    for (int i = 0; plugins[i] != NULL; i++) {
        printf("Загрузка плагина: %s\n", plugins[i]);

        Plugin* plugin = load_plugin(plugins[i]);
        if (plugin == NULL) {
            printf("Не удалось загрузить плагин %s\n\n", plugins[i]);
            continue;
        }

        printf("Загружен плагин: %s v%s\n", plugin->name, plugin->version);

        // Используем плагин
        plugin->initialize();

        // Тестовые данные для каждого плагина
        if (strcmp(plugins[i], "plugin_a") == 0) {
            plugin->process("Hello World from Plugin A!");
        } else if (strcmp(plugins[i], "plugin_b") == 0) {
            plugin->process("10 + 5");
            plugin->process("20 * 3");
        }

        plugin->cleanup();
        unload_plugin(plugin);

        printf("Плагин %s выгружен\n\n", plugins[i]);
    }

    printf("Все плагины протестированы\n");
    return 0;
}
EOF

# 7. Компилируем основную программу
gcc plugin_manager.c -L. -lpluginlib -ldl -o plugin_manager
```



```
# 8. Запускаем менеджер плагинов  
export LD_LIBRARY_PATH=.:$LD_LIBRARY_PATH  
./plugin_manager
```